

■ Cours Express — Catégorie 1

■ Pièges & Erreurs Python Fondamentales (Bugs, Async & Structure)

Objectif : Maîtriser les fautes de syntaxe, de gestion mémoire, de concurrence async et de structure logicielle les plus fréquemment testées en Code Review chez Mistral AI.

1. ■ Les Pièges d'Asyncio & des Coroutines

A. Oubli du mot-clé await sur une Coroutine Async

Problème : Appeler une fonction async def sans await renvoie un objet temporaire <coroutine> non exécuté.

Impact Scalabilité : Les opérations E/S ne s'exécutent pas et bloquent le pipeline.

Impact Structure : Rupture du contrat de type de retour entre modules.

■ Code Buggué	■ Correction Propre
<pre>async def fetch_data(user_id: str) -> dict: ... async def process(): data = fetch_data('usr_123') # ■ Manque await ! print(data['name']) # ■ TypeError: 'c oroutine' object is not subscriptable</pre>	<pre>async def process(): data = await fetch_data('usr_123') # ■ Corrigé print(data['name'])</pre>

B. Appel interdit à asyncio.run() dans une Boucle Active (FastAPI)

Problème : asyncio.run() tente d'ouvrir une nouvelle boucle d'événements. Dans un serveur Web async (FastAPI/Uvicorn), une boucle tourne déjà.

Impact Scalabilité : Crashe les workers sous charge avec RuntimeError: Cannot call asyncio.run() from a running event loop.

■ Code Buggué	■ Correction Propre
<pre>@app.post('/predict') async def predict_endpoint(): res = asyncio.run(run_inference()) # ■ Erreur f atale !</pre>	<pre>@app.post('/predict') async def predict_endpoint(): res = await run_inference() # ■ Utiliser await</pre>

2. ■ Mémoire, Portée & Gestion des Ressources

A. Arguments par Défaut Mutables (def fn(items=[]))

Problème : Les arguments par défaut sont instanciés une seule fois au chargement du module. La même liste est réutilisée et polluée entre toutes les requêtes utilisateurs.

Impact Structure : Effet de bord global et violation de l'isolement des sessions.

■ Code Buggué	■ Correction Propre
<pre>def append_msg(msg: str, history: list = []) -> lis t: history.append(msg) # ■ Pollue l'état global return history</pre>	<pre>def append_msg(msg: str, history: Optional[List[str]] = None) -> List[str]: if history is None: history = [] history.append(msg) return history</pre>

B. Fuite de Ressources (Fichiers/Sockets sans Context Manager)

Problème : Ouvrir des fichiers/connexions sans bloc with ou async with.

Impact Scalabilité : Épuisement des descripteurs de fichiers OS (Too many open files) lors d'erreurs récurrentes.

■ Code Buggué	■ Correction Propre
<pre>def load_config(path: str): f = open(path, 'r') data = json.load(f) # ■ Si exception, f n'est j amais fermé ! return data</pre>	<pre>def load_config(path: str): with open(path, 'r') as f: # ■ Fermeture garant ie return json.load(f)</pre>

3. ■ Exceptions & Architecture de Code

A. Exceptions Ravalées Silencieusement (except: pass)

Problème : Capturer Exception en faisant pass masque la dégradation du système et rend le débogage impossible.

■ Code Buggué	■ Correction Propre
<pre>try: res = call_mistral_api(prompt) except Exception: pass # ■ Le programme continue avec un état cor rompu</pre>	<pre>try: res = call_mistral_api(prompt) except Exception as e: logger.exception('Échec API Mistral') raise APIError('Erreur inférence') from e</pre>

■ Réflexes Oraux pour l'Entretien Mistral (Jour J)

1. 'Je vois `async\_fn()`, il manque `await`' → sinon on manipule un objet Coroutine au lieu de la donnée.
2. 'Pas de `asyncio.run()` dans FastAPI' → nous sommes déjà dans l'Event Loop active du serveur.
3. 'Jamais `items=[]` en valeur par défaut' → piège de mutabilité partagée entre toutes les requêtes.
4. 'Toujours utiliser `with` / `async with`' → garantit la libération déterministe des sockets et fichiers.
5. 'Bannir `except: pass`' → casse la chaîne d'observabilité et le p99 latency monitoring.